ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Д.О. Шевяков
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ ИНТЕРНЕТА ВЕЩЕЙ И ВЕБ-ИНТЕРФЕЙСА

по курсу: ИНТЕРНЕТ ВЕЩЕЙ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №	4321		Д.Ю. Бычков
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Приобретение навыков проектирования приложения интернета вещей и интерфейса для отображения состояния системы.

2 Задание

Вариант задания: Умный дом.

1. В рамках предложенной (или своей) темы спроектируйте систему, состоящую из 4-5 уникальных классов. Подумайте имеют ли классы общие методы и свойства, если таковые есть, выделите их в абстрактный класс. Обозначьте связь элементов системы (Подсказка: не лишним будет создать отдельный объект, который не имеет физического воплощения, а отвечает только за обработку и передачу данных между другими объектами, если это необходимо, в моём примере – это MainControlUnit). Подумайте какие данные в системе будет хранить база данных, обозначьте её отдельной сущностью, связями обозначьте какие данные и с каких вещей она должна хранить. Поскольку следующие лабораторные подразумевают написание кода на Python, в котором модификаторы доступа носят рекомендательный характер, ими можно пренебречь. (Это я про «+», «-» и «/» на диаграмме)

2. В рамках предложенной (или своей) темы спроектируйте, как минимум два интерфейса. Два интерфейса могут быть нужны, например, для пользователя и администратора. На примере умного дома это может быть интерфейс ребёнка, который только отображает некоторые данные и позволяет управлять некоторыми из вещей и интерфейс родителя, которые может позволять более глубоко настраивать управление домом. Подумайте какие данные лучше скрыть от одного пользователя, какие лучше показать, как лучше их отобразить.

3 Результат проделанной работы

3.1 Диаграмма классов

В ходе выполнения поставленной задачи была создана диаграмма классов, показанная на рисунке 1.

Диаграмма была загружена на облачное хранилище для просмотра в лучшем качестве. Ссылка на хранилище: https://disk.yandex.ru/i/AVCmxdFV3SMOaQ .

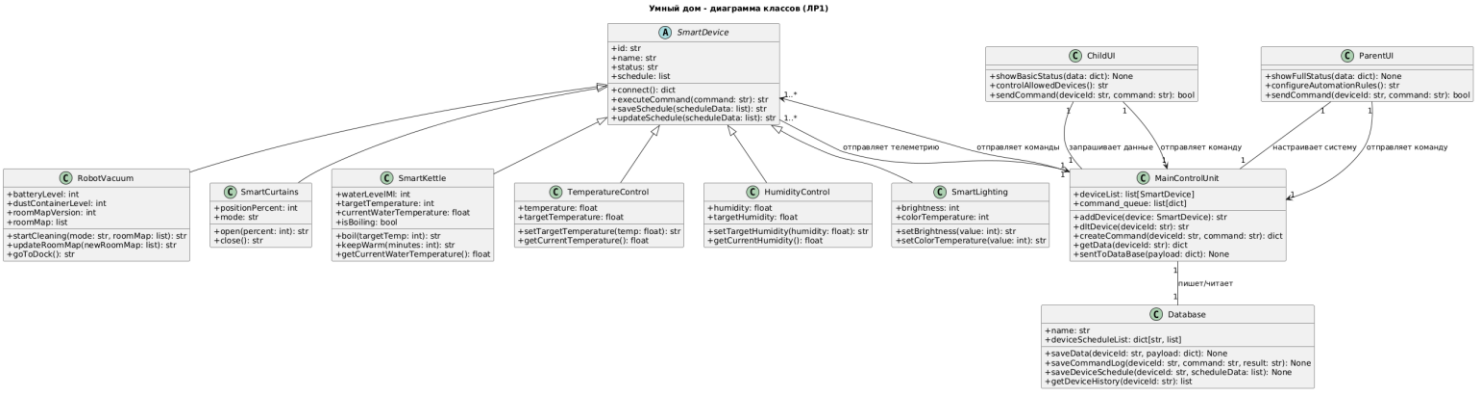

Рисунок 1 — Диаграмма классов

Описание созданных классов:

Базовый абстрактный класс SmartDevice задает общую структуру для всех устройств умного дома. Класс содержит основные поля: id для уникальной идентификации, name для отображаемого имени устройства, status для текущего состояния и schedule для хранения расписания работы. Методы connect(), executeCommand(), saveSchedule() и updateSchedule() обеспечивают единый интерфейс подключения, выполнения команд и управления расписанием для всех дочерних устройств.

Класс RobotVacuum наследуется от SmartDevice и реализует логику робота-пылесоса. Поля batteryLevel, dustContainerLevel, roomMapVersion и roomMap описывают заряд, заполненность контейнера и актуальную карту помещений. Методы startCleaning(), updateRoomMap() и goToDock() позволяют запускать уборку по режимам, обновлять карту и возвращать устройство на док-станцию.

Класс SmartCurtains также наследуется от SmartDevice и описывает умные шторы. Поле positionPercent хранит текущий процент открытия, а mode определяет режим работы (например, ручной или автоматический). Методы open() и close() управляют положением штор.

Класс SmartKettle наследуется от SmartDevice и моделирует умный чайник. Поля waterLevelMl, targetTemperature, currentWaterTemperature и isBoiling позволяют

отслеживать объем воды, целевую и текущую температуру, а также процесс кипячения. Методы boil(), keepWarm() и getCurrentWaterTemperature() обеспечивают нагрев, поддержание температуры и показ текущей температуры воды.

Класс TemperatureControl наследуется от SmartDevice и отвечает за управление температурой в помещении. Поля temperature и targetTemperature содержат текущие и целевые показатели. Методы setTargetTemperature() и getCurrentTemperature() используются для настройки микроклимата и получения актуальных данных с датчиков.

Класс HumidityControl наследуется от SmartDevice и управляет влажностью воздуха. Поля humidity и targetHumidity отражают текущий и требуемый уровень влажности. Методы setTargetHumidity() и getCurrentHumidity() позволяют поддерживать комфортные параметры среды.

Класс SmartLighting наследуется от SmartDevice и реализует управление освещением. Поля brightness и colorTemperature задают яркость и цветовую температуру света. Методы setBrightness() и setColorTemperature() позволяют настраивать освещение под разные сценарии: чтение, отдых, ночной режим.

Класс MainControlUnit является центральным узлом системы умного дома. Поле deviceList хранит список подключенных устройств, а command_queue — очередь команд на выполнение. Методы addDevice(), dltDevice(), createCommand(), getData() и sentToDataBase() обеспечивают регистрацию устройств, формирование команд, получение телеметрии и передачу данных в базу.

Класс Database отвечает за долговременное хранение информации. Поле deviceScheduleList хранит расписания устройств, а также ведутся истории данных и команд. Методы saveData(), saveCommandLog(), saveDeviceSchedule() и getDeviceHistory() позволяют сохранять телеметрию, логи управления и получать историю работы каждого устройства.

Классы ChildUI и ParentUI представляют пользовательские интерфейсы с разными правами доступа. ChildUI ориентирован на базовый просмотр статуса (showBasicStatus()) и управление разрешенными устройствами (controlAllowedDevices(), sendCommand()). ParentUI предоставляет полный контроль: отображение расширенного состояния (showFullStatus()), настройку автоматизаций (configureAutomationRules()) и отправку команд (sendCommand()).

По связям в системе: все устройства (SmartDevice) отправляют телеметрию в MainControlUnit, а центральный блок отправляет им команды; MainControlUnit взаимодействует с Database для записи и чтения данных; интерфейсы ChildUI и ParentUI

работают через центральный блок, запрашивая состояние и отправляя управляющие команды.

3.2 Интерфейсы

В разработанной системе умного дома реализованы два интерфейса с разграничением прав доступа: пользовательский и административный. Они продемонстрированы на рисунках 2-3.

Пользовательский интерфейс предназначен для повседневного управления устройствами и получения информации о текущем состоянии дома, тогда как административный интерфейс обеспечивает расширенные возможности конфигурирования системы.

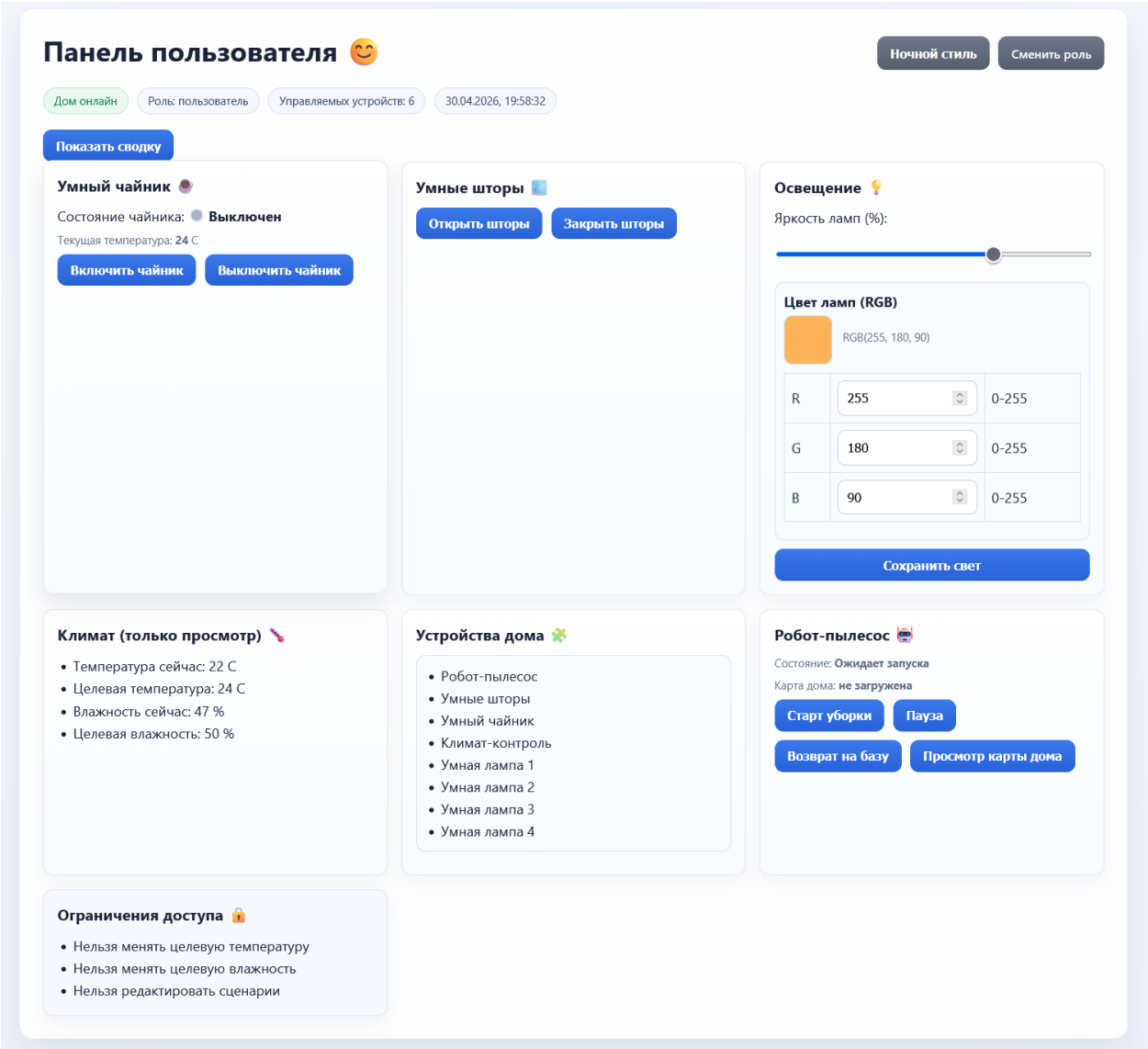

Рисунок 2 — Интерфейс пользователя

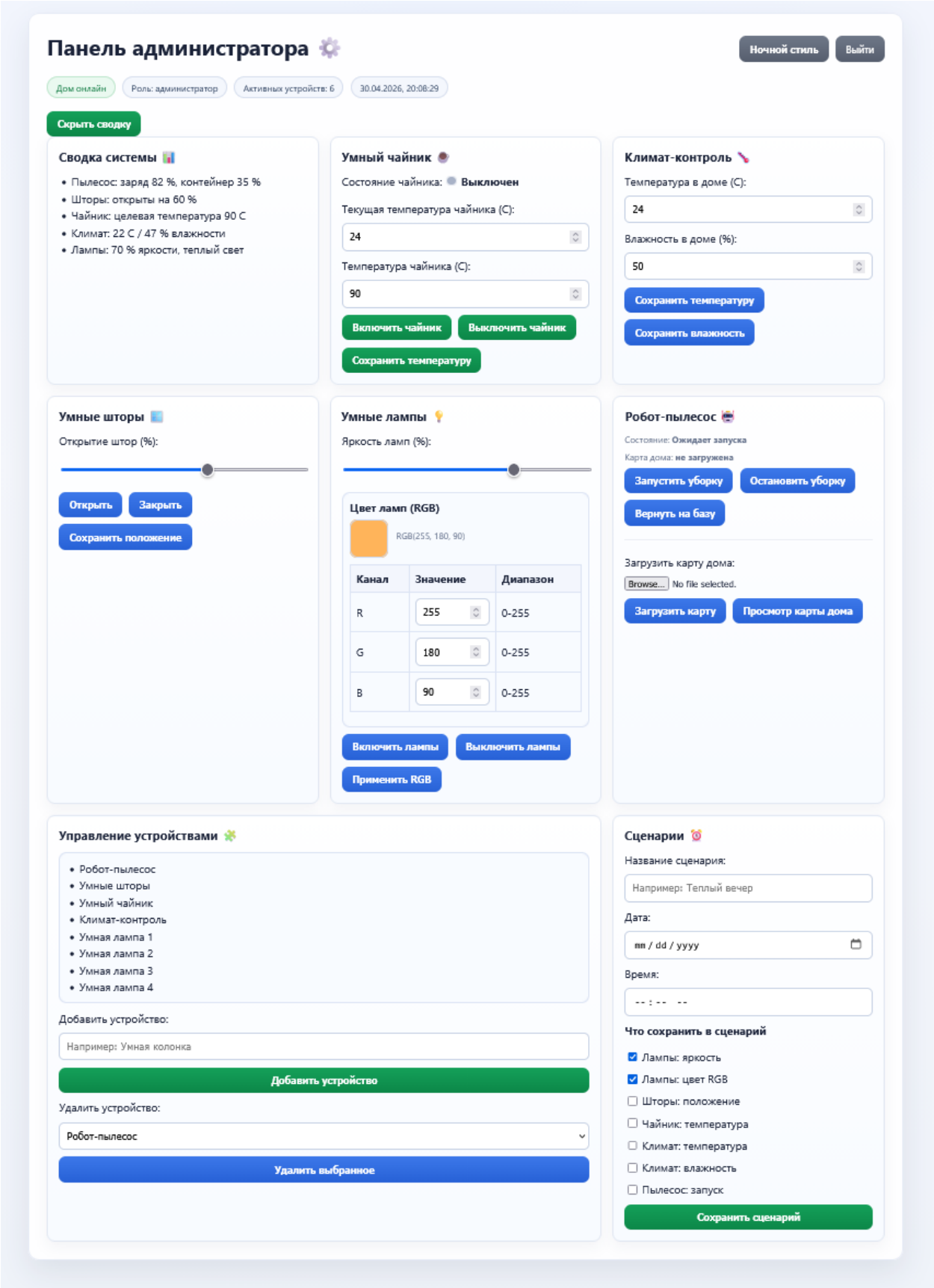

Рисунок 3 — Интерфейс администратора

4 Вывод

В ходе выполнения лабораторной работы были приобретены навыки проектирования приложения интернета вещей и интерфейса для отображения состояния систем.

ПРИЛОЖЕНИЕ А

Листинг кода для формирования диаграммы классов в plantUML:

```
@startuml
skinparam classAttributeIconSize 0
title Умный дом - диаграмма классов (ЛР1)

abstract class SmartDevice {
  +id: str
  +name: str
  +status: str
  +schedule: list
  +connect(): dict
  +executeCommand(command: str): str
  +saveSchedule(scheduleData: list): str
  +updateSchedule(scheduleData: list): str
}

class RobotVacuum {
  +batteryLevel: int
  +dustContainerLevel: int
  +roomMapVersion: int
  +roomMap: list
  +startCleaning(mode: str, roomMap: list): str
  +updateRoomMap(newRoomMap: list): str
  +goToDock(): str
}

class SmartCurtains {
  +positionPercent: int
  +mode: str
  +open(percent: int): str
  +close(): str
}
```

```
class SmartKettle {
  +waterLevelMl: int
  +targetTemperature: int
  +currentWaterTemperature: float
  +isBoiling: bool
  +boil(targetTemp: int): str
  +keepWarm(minutes: int): str
  +getCurrentWaterTemperature(): float
}

class TemperatureControl {
  +temperature: float
  +targetTemperature: float
  +setTargetTemperature(temp: float): str
  +getCurrentTemperature(): float
}

class HumidityControl {
  +humidity: float
  +targetHumidity: float
  +setTargetHumidity(humidity: float): str
  +getCurrentHumidity(): float
}

class SmartLighting {
  +brightness: int
  +colorTemperature: int
  +setBrightness(value: int): str
  +setColorTemperature(value: int): str
}

class MainControlUnit {
  +deviceList: list[SmartDevice]
  +command_queue: list[dict]
  +addDevice(device: SmartDevice): str
```

```
  +dltDevice(deviceId: str): str
  +createCommand(deviceId: str, command: str): dict
  +getData(deviceId: str): dict
  +sentToDataBase(payload: dict): None
}

class Database {
  +name: str
  +deviceScheduleList: dict[str, list]
  +saveData(deviceId: str, payload: dict): None
  +saveCommandLog(deviceId: str, command: str, result: str): None
  +saveDeviceSchedule(deviceId: str, scheduleData: list): None
  +getDeviceHistory(deviceId: str): list
}

class ChildUI {
  +showBasicStatus(data: dict): None
  +controlAllowedDevices(): str
  +sendCommand(deviceId: str, command: str): bool
}

class ParentUI {
  +showFullStatus(data: dict): None
  +configureAutomationRules(): str
  +sendCommand(deviceId: str, command: str): bool
}

SmartDevice <|-- RobotVacuum
SmartDevice <|-- SmartCurtains
SmartDevice <|-- SmartKettle
SmartDevice <|-- TemperatureControl
SmartDevice <|-- HumidityControl
SmartDevice <|-- SmartLighting

MainControlUnit "1" -- "1" Database : пишет/читает
```

```
SmartDevice "1..*" --> "1" MainControlUnit : отправляет телеметрию
MainControlUnit "1" --> "1..*" SmartDevice : отправляет команды

ChildUI "1" -- "1" MainControlUnit : запрашивает данные
ParentUI "1" -- "1" MainControlUnit : настраивает систему
ChildUI "1" --> "1" MainControlUnit : отправляет команду
ParentUI "1" --> "1" MainControlUnit : отправляет команду

@enduml
```